

IoT Crypto: An end-to-end encryption protocol for satellite-based LoRaWAN

Vlad Dancau
EPFL

vlad.dancau@epfl.ch

Simon Reding
WiSeKey S.A.

sreding@wisekey.com

Gabriel Laupré
Space Campus

EPFL

gabriel.laupre@epfl.ch

Haitham Al Hassanieh

Laboratory of Sensing and Networking Systems

EPFL

haitham.alhassanieh@epfl.ch

Abstract—IoT Crypto is a new cryptographic protocol designed to work on top of the LoRaWAN Medium Access Control (MAC) stack. It has specifically been designed to provide confidentiality, integrity and authenticity for messages transmitted through low Earth orbit (LEO) satellites. It also offers robust defenses against physical key compromise, while providing a flexible rekeying mechanism, throughout very high latency links. Additionally, the protocol is intended to be used in conjunction with SEALSQ VaultIC series cryptographic chips, while being compliant with the algorithm requirements of the FIPS 140-3 standard, allowing government systems to transmit data using IoT Crypto.

I. INTRODUCTION

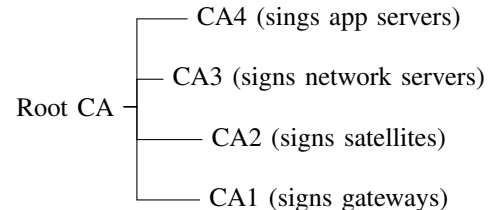
The LoRaWAN MAC protocol is built on top of the LoRa radio modulation technique, guaranteeing the passage of circuit-switched packet data between LoRaWAN gateways and network servers. The LoRaWAN architecture IoT Crypto builds upon is structured in the following way: devices are connected to a gateway through a physical (assumed to be secure) medium, gateways then connect to satellites through LoRa. Any data payload transmitted by a gateway is stored on a satellite's buffer, and pulled from it as soon as a satellite passes over a ground station. Each ground station is then connected through a standard authenticated channel (TLS) to application servers, the objective being to transmit end-to-end encrypted data from a device to a specific application server. IoT Crypto allows for an unlimited number of security associations between an application server and devices, each device being limited to a single association with an application server. In this sense, IoT Crypto provides end-to-end encryption, integrity and authenticity to data transmitted from a device to an application server. It also offers forward secrecy, meaning that if a device's cryptographic keys are compromised at time t , an adversary shall not be able to decrypt messages sent in the past. IoT Crypto also provides anonymous routing of data, meaning that an adversary shall not be able to determine which device is sending data. Our protocol also allows interoperability with other mediums by making use of the common ASN.1 interface description language [3, 4], allowing interoperability with 3GPP networks; from a cryptographic perspective, IoT Crypto only uses FIPS 140-3 approved primitives, in accordance with the NSA CSfC framework, while building upon a PKI architecture. All the interactive cryptographic considerations in this report will be

made in the Dolev-Yao [7] model where an adversary can overhear, intercept, and synthesize any message.

II. GATEWAY PROVISIONING

A. Public Key Infrastructure

As stated in the Introduction, IoT Crypto makes use of the VaultIC series cryptographic chips to accelerate the execution of cryptographic primitives on hardware, while remaining compliant with the FIPS 140-3 approved algorithms [19], in this sense we choose to force the cryptographic primitives used by IoT Crypto, eliminating cryptographic agility, a practice encountered in other stateless cryptographic protocols such as Wireguard [8]. This choice is motivated by the fact that if a FIPS 140-3 cryptographic primitive is broken, then hardware needs to be completely replaced, thus making no sense to allow a dynamic negotiation of non-standard cryptographic schemes, which would only increase the attack surface for potential adversaries. Accordingly, IoT Crypto uses the following cryptographic primitives: AES-256 [16] for symmetric encryption, GCM [23] as an AES mode of operation, elliptic curve P-384 for asymmetric cryptography, which we selected over RSA for its shorter key lengths and higher power efficiency, for the same level of security [21]. Our protocol also makes use of the SHA-384 hash function [18], the ECDH elliptic curve Diffie-Hellman scheme [1] and finally the ECDSA elliptic curve digital signature scheme [17]. As such, IoT Crypto requires a Public Key Infrastructure (PKI) structure to be established, according to the tree that follows.



In this sense, a root certificate authority (CA) signs various sub-CAs, which in turn sign identity certificates associated to the various components of IoT Crypto; this project does not cover the protections in case of compromise of any CA's private keys, which we assume to be persistently secret.

B. Factory Provisioning of Gateways

IoT Crypto is an authenticated protocol: before any gateway is allowed to send data to a network server, through a satellite, it needs to be provisioned. This step is achieved through the usage of long term keys that are hardcoded to gateways when manufactured, or programmed by customers. During this step, a gateway is powered on for the first time, proceeding to generate an elliptic curve P-384 key pair locally (following the principle that private keys shall never leave their execution environment). It then proceeds to send its fresh public key to the associated PKI server, which signs and saves it, sending back its ECDSA signature to the gateway. The gateway then derives and stores an ephemeral identity made of the first 8 bytes of the SHA-384 hash of the public key. The gateway is then ready to be delivered through an uncontrolled supply chain, where an adversary is assumed to have the capability to change the public/private key pair, but not to read it, as it is stored on a tamper-proof memory section of a VaultIC series cryptographic processor.

C. Satellite Provisioning of Gateways

After a gateway is fielded, its administrator proceeds to inform its associated network server of that fact through a side channel (for e.g. a web application), along with its fixed geographical coordinates. A scheduler then computes the batch of gateways to be fielded at a discrete list of timestamps. When a specific timestamp is reached, the scheduler crafts the packet in fig. 1, which we denote by the *OTA_Provision* identifier. In

MID 4b	Network Server PK 384b	Nonce 32b
Signature 1 768b	Signature 2 768b	

Fig. 1. *OTA_Provision* packet

this *OTA_Provision* packet, the message ID (MID) represents a uniform message identifier throughout the protocol. It is aimed at increasing the decoding efficiency, especially on embedded hardware. The usage of message identifiers on circuit-switched networks is a property with proven reliability advantages, exemplified by the SCIP and Firefly protocols [9, 15] developed by the U.S. National Security Agency to provide secure data, voice and key management across tactical networks for the U.S. and its allies. Later, the network server public key is included in the *OTA_Provision* packet; we assume that gateways shall have the agility to be provisioned and communicate with any cryptographic certificate signed by its associated certificate authority (CA 3 in this case). This allows for redundancy in case of key compromise of the network server's keys, to have multiple network servers for different security groups/customers, and firsthand to periodically rotate the network server key pairs. The packet then contains a 32-bit nonce; this nonce is persistently kept in memory for both the gateway and the network server and is simply a

counter that keeps track of the number of messages sent between the gateway and the network server. Then, the packet contains two ECDSA signatures (made using curve P-384). The first one is the signature of the network server public key by the certificate authority n°3 and allows the gateway to guarantee that the network server is authenticated to have authority over it. Additionally, the second signature signs the following hash: SHA-384 (MID || Network Server PK || Signature 1 || Nonce), [||] being the concatenation operator for strings. When a gateway receives such a packet, it lazily verifies the identity of the network server by performing an ECDSA signature verification of the 1st signature, and then proceeds to check the second signature, which ensures that the request, namely to initialize and provision the gateway, identified by its specific MID, is authentic and has been received integrally. The integration of the message counter nonce into the 2nd signature guarantees that provisioning messages cannot be replayed by an adversary as they would violate the saved counter. After a gateway has successfully verified this message, it proceeds to generate a new P-384 key pair and a random 96 bits nonce, crafting the packet in fig. 2 and 3.

MID 4b	GW Nonce 96b
Encrypted payload 448b	GCM Auth Tag 128b

Fig. 2. *OTA_Provision_Reply* packet

Ephemeral Id 64b	New Public Key 384b
----------------------------	-------------------------------

Fig. 3. *OTA_Provision_Reply* encrypted payload

At this point, the gateway computes an Elliptic Curve Diffie-Hellman (ECDH) [1] shared initial key that we denote $K^{\text{Init}} = \text{ECDH}(\text{Factory Secret Key}, \text{Network Server Public Key})$, and derives a gateway secret key using an AES-256-based key derivation function (KDF) [5] such that $K^{\text{gateway}} = \text{KDF}(K^{\text{Init}}, \text{Gateway Nonce} || \text{Gateway Ephemeral ID})$. [13] It then encrypts the *OTA_Provision_Reply* payload using the AES-256-GCM authenticated encryption with associated data (AEAD) cipher [23], containing the newly generated public key of the gateway, authenticating the nonce and the MID. Reciprocally, the network server that initiated the provisioning request computes the same shared secrets as the gateway using the symmetric Diffie-Hellman function. Upon receiving a new public key from a gateway it requested a provisioning, the server stores it in a central database, and proceeds to sign it using CA 1's private key, signature which it also saves, crafting a rekey confirmation packet depicted in fig. 4.

The objective of this new packet is to confirm to the gateway that it can fully rekey using its new key pair,

MID 4b	NS Nonce 96b
Encrypted payload Variable	GCM Auth Tag 128b

Fig. 4. OTA_Provision_Reply_Acknowledge packet

Status Variable ($\approx 100b$)	New PK Signature 768b (optional)
--	--

Fig. 5. OTA_Provision_Reply_Acknowledge encrypted payload

re-computing the shared secrets of the security association between it and the network server. The status field contains an administrative message for the gateway, in this situation `provision:accept` or `provision:deny` depending on the completion of the cryptographic checks on the network server. As an additional security measure on top of the AEAD payload, the network server can also send the signature of the gateway's new public key.

D. Periodic Rekeying of Gateways

Following the recommendations of the FIPS 140-3 standard [19], we can only encrypt at most $2^{32} - 1$ messages using the same key. In this sense, our shared message counter that the gateway and network server increment at each exchanged message triggers when reaching a value close to its limit of 32 bits a conventional rekey of the gateway, through an already existing authenticated channel. Using the same packet structure as OTA_Provision_Reply, the network server sends the following encrypted payload in fig. 6 to the gateway to be rekeyed.

Command Variable ($\approx 100b$)	Command Data 384b for PK (optional)
---	---

Fig. 6. OTA_Rekey encrypted payload

In this packet, a `rekey-request` command is crafted, optionally including a new network server public key (if it has changed). The packet is then authenticated using the aforementioned process in the OTA_Provision_Reply procedure.

III. KEY MANAGEMENT AND END-TO-END ENCRYPTION FOR DEVICES

A. Introduction

As stated in the introduction, the IoT Crypto gateways only concentrate and transmit data that devices encrypt on an end-to-end basis up to application servers, logically located behind the network server(s). Each application server has an associated cryptographic certificate signed by the certificate authority n°4. It allows both the authentication of an application server to a network server, through standard protocols such as TLS (not covered by IoT Crypto), but also the establishment of security associations with end devices.[6]

B. Device Provisioning

Identically to the gateways, devices are registered through a side-channel to the network server, and linked to a single gateway. Using the same scheduling system mentioned in part C., the network server crafts a bulk packet that provisions and rekeys devices associated to a single gateway, pictured in fig. 7.

MID 4b	NS Nonce 96b
Encrypted payload N * 384b	GCM Auth Tag 128b
Bulk Certificate Data: map {deviceId} -> {AS PK} N * 384b	

Fig. 7. OTA_Device_Rekey encrypted payload

The bulk certificate data field contains a map that associates a unique device identifier (across the entire network) and the public key of the application server to which this device is intended to communicate. This packet is encrypted over the gateway administrative channel, as any other rekey message. Upon reception of this packet by a gateway, a rekey message (not covered by IoT Crypto) is sent to each targeted device, which triggers it to generate a fresh P-384 key pair. Each device then computes an initial secret key $K^{\text{Device}, 1} = \text{ECDH}(\text{Device Secret Key}, \text{App Server Public Key})$.

C. Device Data Transmission

Devices use the same cryptographic constructs as gateways, to allow interoperability and an uniform hardware management. We shall mention that each device keeps a local counter denoted "device counter" that counts each message sent by a device to the gateway, with the application server as a destination. The device that sends data then concatenates its device ID and internal counter. The device then requests the gateway the latest cryptographic nonce that the gateway has in memory. Once received, the end device computes a new nonce = $\text{KDF}(K^{\text{Device}}, \text{Gateway Nonce})$, having a length of 96 bits. It then proceeds to transmit this plaintext packet to the gateway, together with the newly computed nonce. The gateway in turn encrypts the plaintext packet (fig. 8) using AES-256-CTR with its latest shared secret (with the network server), while using the new nonce received from the device.

Device Id 64b	Device Counter 32b
-------------------------	------------------------------

Fig. 8. Cryptographic nonce plaintext

Thereafter, the gateway transmits back the encrypted cryptographic nonce to the device, that will later use it as a nonce in the data encryption process.

MID 4b	Crypto Nonce 96b
Encrypted Data Variable	GCM Auth Tag 128b

Fig. 9. Cryptographic nonce plaintext

At this point, when a device needs to send data, it encrypts using AES-256-GCM its ASN.1 plaintext, using the newly generated cryptographic nonce that it received from the gateway, and proceeds to broadcast it to its gateway, for transmission to the gateway's network server and later on to device's application server.

The main advantage of our device cryptographic nonce is that it integrates an encrypted device ID that only the network server can decrypt, allowing an anonymized routing of device data to its corresponding application server, while satisfying the uniqueness requirement of one nonce per message of GCM [12]. It also reduces the overhead of a data packet by 30% by eliminating the need for a device ID field.

D. Forward Secrecy for Device Data

As the devices and gateways are assumed to be unsupervised, having long deployment periods (5-10 years), there is a possibility that devices could be compromised, and cryptographic keys extracted from them. Nevertheless, an adversary shall not be capable of decrypting recorded messages from the past for a device using the material he extracted from it. When it comes to the gateways, we assume that their hardware can detect tampering, and automatically erase all data when it occurs, as tampering a gateway would allow an adversary to forge future messages. [10]

In order to provide forward secrecy, IoT Crypto employs a key ratcheting [2] mechanism where each device derives a new key after a message has been sent by chaining together AES-based key derivation functions. In this sense, 256-bit shared secrets are derived based on the latest gateway nonce and device counter, taking as a secret key input for the KDF the latest shared secret present on a device. After a message has been sent, the device deletes the old key and only keeps the KDF-derived shared secret, repeating the process at each iteration (fig. 10).

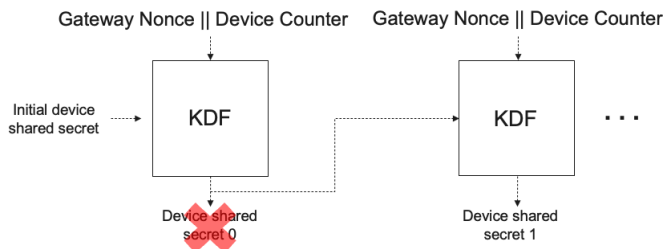


Fig. 10. IoT Crypto Key Ratchet

IV. PERFORMANCE ANALYSIS

A. Introduction

Because IoT Crypto was designed with the LoRaWAN over satellite transmissions medium in mind, which provides a maximum throughput of 10 kbps per satellite, adding to that latencies of up to 2 hours for small low Earth orbit constellations, our main goals for this project have been to minimize the amount of overhead that our packet structures add to transmitted data, the number of round trips necessary for key management purposes, and finally ensuring interoperability with future systems.

B. Round-Trip Performance

IoT Crypto achieves key management for gateways and devices in 1.5 round trips, it can also completely rekey a gateway or a device group in 1 round trip.

C. Packet Overhead Optimization

Most importantly, on a transport packet, which is used to transmit end-to-end encrypted and authenticated data from a device to an associated application server, IoT Crypto only adds 228 bits of overhead to the LoRaWAN protocol, which is constant among other key management packet types, where most of the transmitted data consists of standard-length P-384 public keys.

D. Interoperability with 3GPP Networks

IoT Crypto packets are entirely encoded using the ASN.1 interface description language, allowing an offloading of certain cryptographic operations, such as rekeys, over conventional 3GPP networks, which could be periodically available in isolated areas for gateways, letting the scarce satellite bandwidth for device data.

V. SECURITY ANALYSIS

A. Introduction

During the design of IoT Crypto, we have tried extensively to verify the protocol using Tamarin [20], a formal verification software that would have allowed us to prove simple security properties of our protocol. Unfortunately, and because of time constraints, we have not managed to formalize IoT Crypto in the Tamarin syntax. Nonetheless, we will perform a formal security analysis of the different key management patterns of IoT Crypto using Noise [11], a library of formally verified cryptographic handshakes.

B. Gateway Provisioning

Because the formal verification of complex protocols based on a public key infrastructure is an open area of research, we will assume that the fundamental authenticity and integrity guarantees that the IoT Crypto PKI subsystem offers are valid. When a server sends a provisioning packet to a gateway, it follows the Noise IKPs2 handshake pattern, where the first packet coming from the network server to the gateway is authenticated to come from the network server (signature n^2), while guaranteeing the identity of this aforementioned

network server using signature 1. In case of missed authentication, where a packet was lost for example, the nonce, which we recommend to be a simple counter starting from a random value, guarantees that an adversary cannot replay, or manipulate the order of provisioning packets, as that would invalidate the 2nd signature. Additionally, this message is not resistant to the compromise of the network server's keys, nor the private keys of the associated certificate authority, unless a certificate revocation list has been uploaded (through a side channel) to the gateway, before provisioning it. Then, when the gateway replies to the request, it benefits from authenticity, integrity and confidentiality, unless its factory keys have been compromised during the supply of the hardware, which is guaranteed not to happen as the VaultIC cryptographic chips for which IoT Crypto has been designed are guaranteed to be tamper-proof. Thereafter, even in the case where an active adversary has determined the shared secret between the gateway and the network server (NS), the NS is guaranteed to have received an authentic and integral new public key from the gateway. Additionally, the AES-based key derivation functions involved during the key management process are guaranteed to generate pseudorandom outputs because they are fed with randomly generated nonces and keys.

C. Gateway and Devices Rekeying

During the rekeying process, both for devices and gateways, we provide forward secrecy for a user-controlled number of rekeys, depending on the required security level, for up to $2^{32} - 1$ messages, while assuming that the gateway shall wipe itself in case of tampering. The avoidance of asymmetric encryption and digital signatures, which would have allowed the usage of ephemeral keys is a deliberate design choice in IoT Crypto, designed to minimize the power consumption of both gateways and end devices.[21] Additionally, the gateway and server derive different traffic encryption keys, using the context-based KDF function which we specified in part II. C., in order to avoid XOR attacks on blocks of ciphertext that could accidentally have been encrypted using the same nonce.

D. Transport Data

The key management of the devices is done over the administrative authenticated encryption with associated data (AEAD) channel of the gateway, backed by the previously enumerated security assumptions and properties. The gateway-backed encryption that constitutes the cryptographic nonce allows the anonymous routing of data from devices to application servers, while the key ratcheting mechanism provides forward secrecy, thus resistance to the compromise of device keys for data that has already been sent. The authenticity of the routing command is guaranteed, as the nonce is part of the associated data authenticated by GCM. Additionally, the usage of AES-256-GCM provides a 256-bit level of security, while ensuring power efficiency [22] and offering an excellent decorrelation between data, as IoT-originated packets generally tend to be highly correlated in their statistical distribution.

VI. FUTURE WORK

In the near future, we aim to deploy IoT Crypto on WISESat, WiseKey's first low Earth orbit constellation. Before achieving this first, we still need to implement the hardware state machine, and an optimized data encoder. [14]

REFERENCES

- [1] Elaine Barker et al. *Recommendation for pair-wise key-establishment schemes using discrete logarithm cryptography*. Tech. rep. NIST SP 800-56Ar3. Gaithersburg, MD: National Institute of Standards and Technology, Apr. 2018, NIST SP 800-56Ar3. DOI: 10.6028/NIST.SP.800-56Ar3. URL: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-56Ar3.pdf> (visited on 10/04/2024).
- [2] Alexander Bienstock et al. "A More Complete Analysis of the Signal Double Ratchet Algorithm". en. In: *Advances in Cryptology – CRYPTO 2022*. Ed. by Yevgeniy Dodis and Thomas Shrimpton. Vol. 13507. Series Title: Lecture Notes in Computer Science. Cham: Springer Nature Switzerland, 2022, pp. 784–813. ISBN: 978-3-031-15801-8 978-3-031-15802-5. DOI: 10.1007/978-3-031-15802-5_27. URL: https://link.springer.com/10.1007/978-3-031-15802-5_27 (visited on 12/24/2024).
- [3] CCSDS. *SPACE DATA LINK SECURITY PROTOCOL*. July 2022. URL: <https://public.ccsds.org/Pubs/355x0b2.pdf>.
- [4] CCSDS. *SPACE DATA LINK SECURITY PROTOCOL— EXTENDED PROCEDURES*. Feb. 2020. URL: <https://public.ccsds.org/Pubs/355x1b1.pdf>.
- [5] Lily Chen. *Recommendation for key derivation using pseudorandom functions*. Tech. rep. NIST SP 800-108r1-upd1. Gaithersburg, MD: National Institute of Standards and Technology (U.S.), Feb. 2024, NIST SP 800-108r1-upd1. DOI: 10.6028/NIST.SP.800-108r1-upd1. URL: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-108r1-upd1.pdf> (visited on 10/03/2024).
- [6] CJ. Tjhai et al. *Multiple Key Exchanges in the Internet Key Exchange Protocol Version 2 (IKEv2)*. en. Tech. rep. RFC9370. RFC Editor, May 2023, RFC9370. DOI: 10.17487/RFC9370. URL: <https://www.rfc-editor.org/info/rfc9370> (visited on 12/26/2024).
- [7] D. Dolev and A. Yao. "On the security of public key protocols". en. In: *IEEE Transactions on Information Theory* 29.2 (Mar. 1983), pp. 198–208. ISSN: 0018-9448. DOI: 10.1109/TIT.1983.1056650. URL: <http://ieeexplore.ieee.org/document/1056650/> (visited on 01/08/2025).
- [8] Benjamin Dowling and Kenneth G. Paterson. "A Cryptographic Analysis of the WireGuard Protocol". en. In: *Applied Cryptography and Network Security*. Ed. by Bart Preneel and Frederik Vercauteren. Vol. 10892. Series Title: Lecture Notes in Computer Science. Cham: Springer International Publishing, 2018, pp. 3–21. ISBN: 978-3-319-93386-3 978-3-319-93387-0. DOI: 10.1007/

- 978-3-319-93387-0_1. URL: https://link.springer.com/10.1007/978-3-319-93387-0_1 (visited on 12/24/2024).
- [9] General Dynamics C4 Systems. *SCIP Signaling Plan*. Dec. 2007.
- [10] Felix Günther et al. “0-RTT Key Exchange with Full Forward Secrecy”. In: *Advances in Cryptology – EUROCRYPT 2017*. Ed. by Jean-Sébastien Coron and Jesper Buus Nielsen. Vol. 10212. Series Title: Lecture Notes in Computer Science. Cham: Springer International Publishing, 2017, pp. 519–548. ISBN: 978-3-319-56616-0 978-3-319-56617-7. DOI: 10.1007/978-3-319-56617-7_18. URL: http://link.springer.com/10.1007/978-3-319-56617-7_18 (visited on 12/24/2024).
- [11] Son Ho et al. “Noise: A Library of Verified High-Performance Secure Channel Protocol Implementations”. In: *2022 IEEE Symposium on Security and Privacy (SP)*. San Francisco, CA, USA: IEEE, May 2022, pp. 107–124. ISBN: 978-1-66541-316-9. DOI: 10.1109/SP46214.2022.9833621. URL: <https://ieeexplore.ieee.org/document/9833621/> (visited on 01/09/2025).
- [12] Information Assurance Directorate. *Galois/Counter Mode (GCM) Data Integrity*.
- [13] Information Assurance Directorate. *Universal ECMQV Key Agreement and TEK Derivation*. Mar. 2010. URL: <https://www.iad.gov/SecurePhone/openDoc.cfm> (visited on 03/10/2024).
- [14] Brian Jalaian et al. “Evaluating LoRaWAN-based IoT devices for the tactical military environment”. In: *2018 IEEE 4th World Forum on Internet of Things (WF-IoT)*. Singapore: IEEE, Feb. 2018, pp. 124–128. ISBN: 978-1-4673-9944-9. DOI: 10.1109/WF-IoT.2018.8355225. URL: <https://ieeexplore.ieee.org/document/8355225/> (visited on 12/24/2024).
- [15] P. Karn and W. Simpson. *Photuris: Session-Key Management Protocol*. en. Tech. rep. RFC2522. RFC Editor, Mar. 1999, RFC2522. DOI: 10.17487/rfc2522. URL: <https://www.rfc-editor.org/info/rfc2522> (visited on 12/26/2024).
- [16] National Institute of Standards and Technology (US). *Advanced Encryption Standard (AES)*. Tech. rep. NIST FIPS 197-upd1. Washington, D.C.: National Institute of Standards and Technology (U.S.), May 2023, NIST FIPS 197-upd1. DOI: 10.6028/NIST.FIPS.197-upd1. URL: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197-upd1.pdf> (visited on 10/03/2024).
- [17] National Institute of Standards and Technology (US). *Digital Signature Standard (DSS)*. Tech. rep. NIST FIPS 186-5. Washington, D.C.: National Institute of Standards and Technology (U.S.), Feb. 2023, NIST FIPS 186-5. DOI: 10.6028/NIST.FIPS.186-5. URL: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.186-5.pdf> (visited on 12/26/2024).
- [18] National Institute of Standards and Technology (US). *Secure hash standard*. Tech. rep. NIST FIPS 180-4. Washington, D.C.: National Institute of Standards and Technology (U.S.), 2015, NIST FIPS 180-4. DOI: 10.6028/NIST.FIPS.180-4. URL: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf> (visited on 01/05/2025).
- [19] National Institute of Standards and Technology (US). *Security requirements for cryptographic modules*. Tech. rep. error: 140-3. Washington, D.C.: National Institute of Standards and Technology (U.S.), 2019, error: 140-3. DOI: 10.6028/NIST.FIPS.140-3. URL: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.140-3.pdf> (visited on 01/05/2025).
- [20] Singam Bhargav Ram and Vanga Odelu. “Security Analysis of a Key Exchange Protocol under Dolev-Yao Threat Model Using Tamarin Prover”. In: *2022 IEEE 12th Annual Computing and Communication Workshop and Conference (CCWC)*. Las Vegas, NV, USA: IEEE, Jan. 2022, pp. 0667–0672. ISBN: 978-1-66548-303-2. DOI: 10.1109/CCWC54503.2022.9720852. URL: <https://ieeexplore.ieee.org/document/9720852/> (visited on 01/09/2025).
- [21] Manuel Suárez-Albela, Paula Fraga-Lamas, and Tiago M. Fernández-Caramés. “A Practical Evaluation on RSA and ECC-Based Cipher Suites for IoT High-Security Energy-Efficient Fog and Mist Computing Devices”. en. In: *Sensors* 18.11 (Nov. 2018), p. 3868. ISSN: 1424-8220. DOI: 10.3390/s18113868. URL: <https://www.mdpi.com/1424-8220/18/11/3868> (visited on 01/05/2025).
- [22] Phithak Thaenkaew, Bruno Quoitin, and Ahmed Meddahi. “Leveraging Larger AES Keys in LoRaWAN: A Practical Evaluation of Energy and Time Costs”. en. In: *Sensors* 23.22 (Nov. 2023), p. 9172. ISSN: 1424-8220. DOI: 10.3390/s23229172. URL: <https://www.mdpi.com/1424-8220/23/22/9172> (visited on 12/26/2024).
- [23] J. Viega and D. McGrew. *The Use of Galois/Counter Mode (GCM) in IPsec Encapsulating Security Payload (ESP)*. en. Tech. rep. RFC4106. RFC Editor, June 2005, RFC4106. DOI: 10.17487/rfc4106. URL: <https://www.rfc-editor.org/info/rfc4106> (visited on 12/26/2024).